# Spark Basics

by Nicola Orecchini

Start: June 2025
Last update: March 2026

# Agenda

Distributed computing

Spark

Infrastructure

Data

API

Runtime

Shuffle deep dive

# Agenda

- Distributed computing

  Spark

  Infrastructure

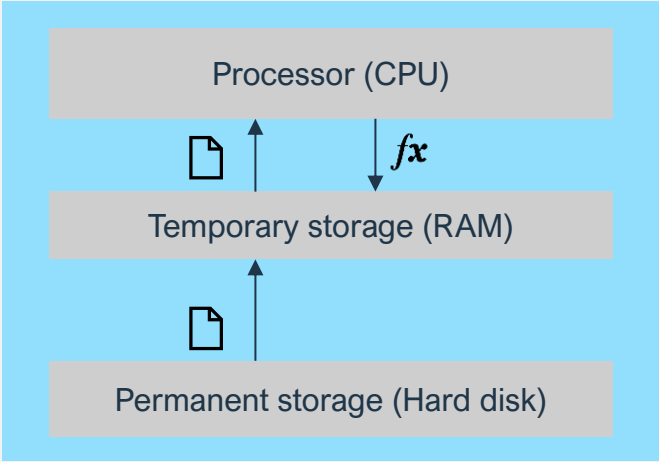
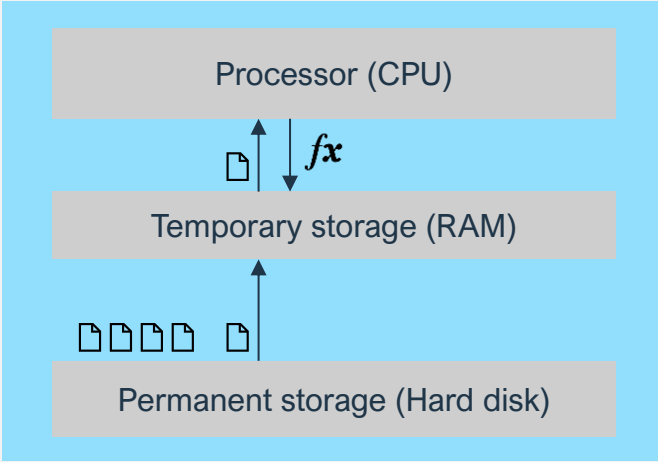
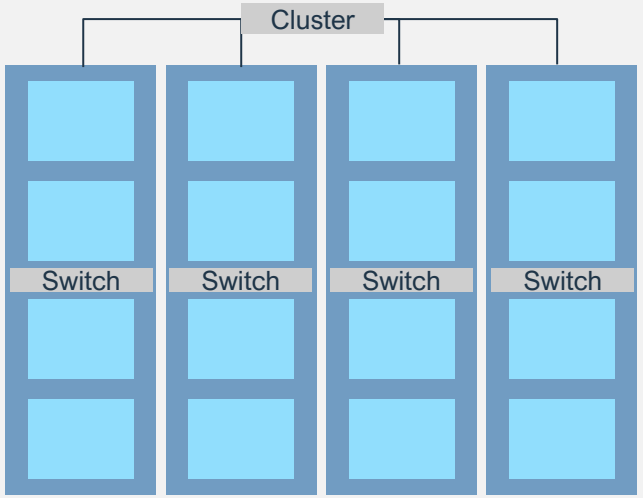
  Data

  API

  Runtime

  Shuffle deep dive

# Applications need new ways to handle big data

| | Single-node, single-read | Single-node, by chunk | Distributed Computing |
|---|---|---|---|
| Application architecture |   |   |   |
| Data processing | • Stores the whole dataset in the RAM | • Reads dataset by chunks<br>• Stores in the RAM only the intermediate statistic value | • Nodes connected by Network<br>• Data transfer between nodes fast<br>• Data transfer between racks limited |
| Data size | • Only small datasets (e.g., 100s of rows) | • Larger datasets, still limited to the HD size (e.g., GB of data) | • Big data datasets (e.g., PB of data, 32 GB per node) |
| How handles Big data | **Vertical scalability**: increase RAM size, non-linear cost increase | | **Horizontal scalability**: add more servers, linear cost increase |

# Distributed computing is the most cost-effective solution, but comes with some challenges

**Parallelization of computation**

- Applications must be coded taking into account that the **dataset** will be **split in chunks** and a local statistic will be computed and then aggregated

**Distributed storage**

- **Data** must be **distributed** in multiple servers, ensuring a **MECE**[1] split

**Node failure**

- Nodes can fail, thus **data replication** must be set (e.g., replicate each chunk of data in 3 different servers)

**Network bottleneck**

- While **data transfer** between nodes of the **same rack** can be **parallelized** (nodes can exchange data in parallel), data transfer **between racks** cannot be parallelized

**Diverse input format**

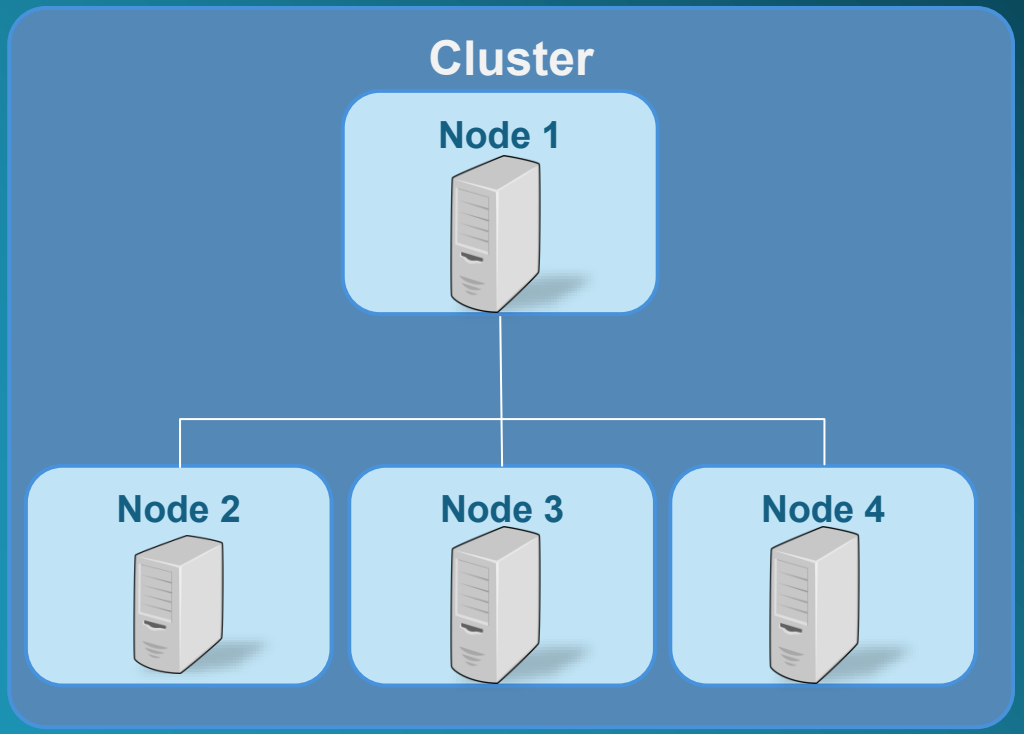
- Different servers could use a **different data format** to produce output, and thus formats need to be reconciled

Managed by programmers

Managed by frameworks (e.g., Spark, Hadoop)

1. MECE: Mutually Exclusive, Collectively Exhaustive. Means no overlaps between dataset partitions and sum of partitions must equal the original dataset

Distributed computing leverages the Cluster of Servers architecture, which works by splitting **data** and **computations** across many machines in a cluster




**Cluster** — A group of machines that work together to perform distributed computations

**Node** — One individual server inside the cluster. It is often a Virtual Machine, i.e. a simulated computer inside a physical server (including RAM, CPU, disk, OS, etc. – like a regular computer)

Source: https://www.friktoria.com/en/what-is-cluster-of-servers.html

# Agenda

Distributed computing

- Spark
  - Infrastructure
  - Data
  - API
  - Runtime
  - Shuffle deep dive

# Agenda

Distributed computing

Spark

- Infrastructure

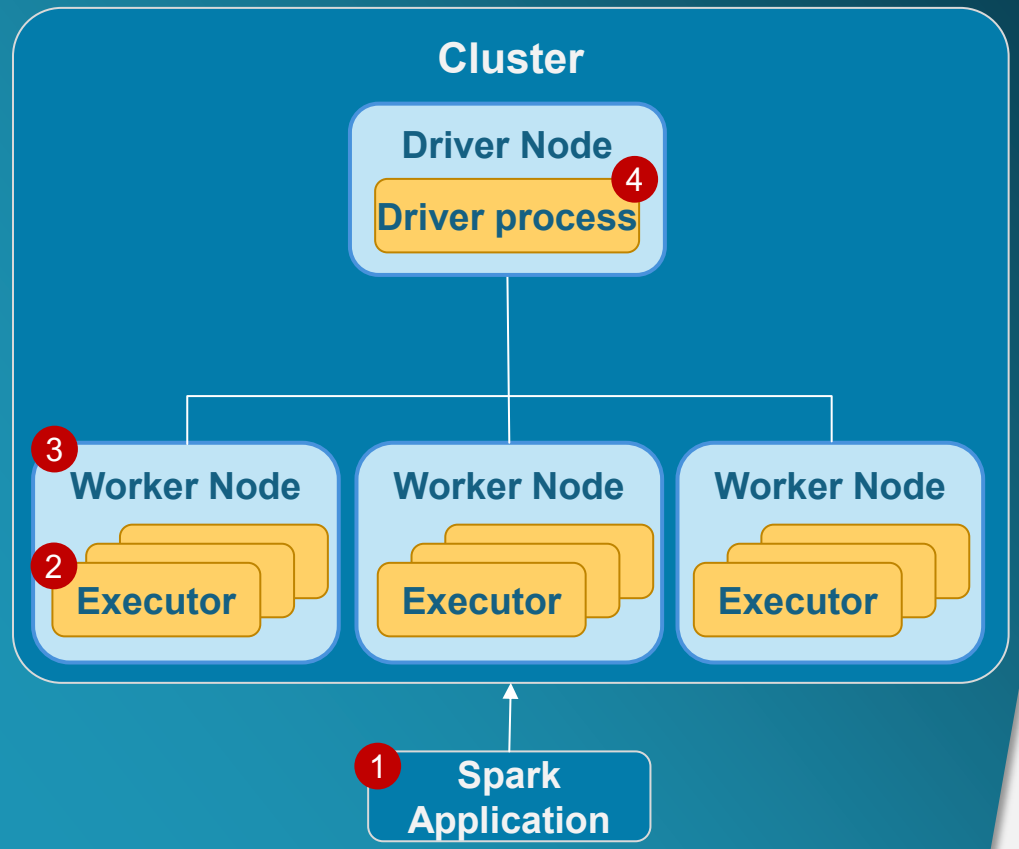
Data

API

Runtime

Shuffle deep dive

# Apache Spark is a distributed computing framework that works by splitting computations across many machines in a cluster




## Spark applications[1] run as independent sets of executor processes[2] on nodes[3] of a cluster, coordinated by a driver[4]

1 **Spark Application** — A program that runs computations on large datasets

2 **Executor** — Independent distributed process that runs computations on cluster nodes and stores data for your application

- runs multiple tasks in parallel on separate threads
- caches data to speed up processing
- communicates results back to the driver

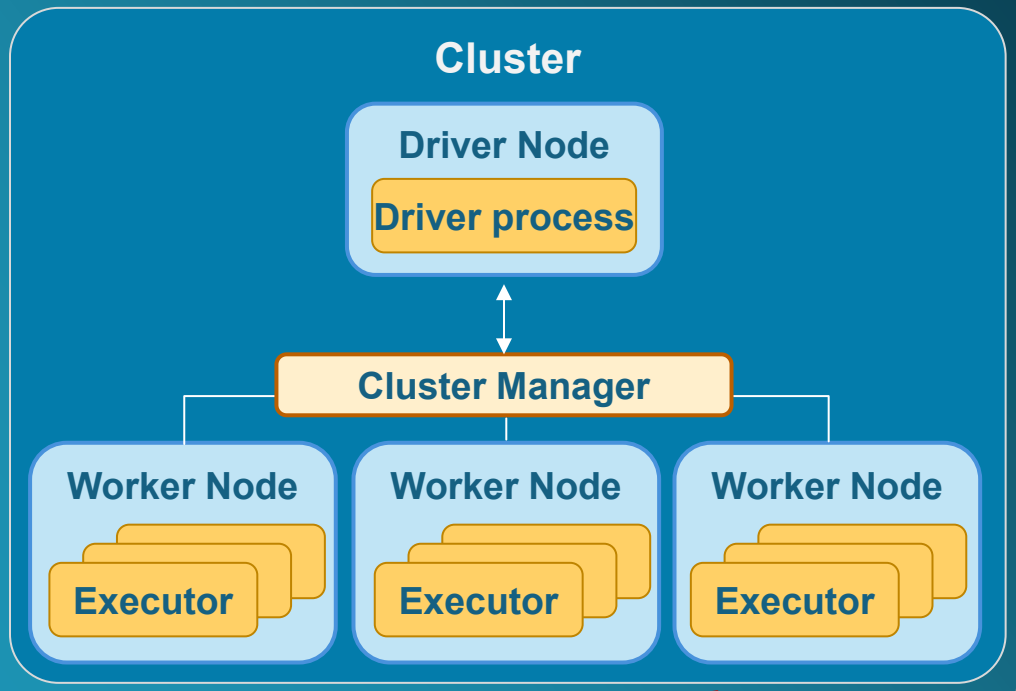
3 **Node** — One individual server inside the cluster

**Worker Node** — A node capable of running executor processes

**Driver Node** — A node capable of running driver processes

4 **Driver** — Process that builds the execution plan for the application and distributes the workload to executors

# A Cluster Manager is responsible for allocating resources in the cluster and launching executors across the nodes




The driver knows **what work needs to be done.** The **Cluster Manager** decides **where that work can run**

## Why?

A cluster typically runs **multiple applications simultaneously**.

The cluster manager is a separate component that ensures:

- **resources** are **shared** across applications
- **nodes** are efficiently **utilized**
- Spark **applications** can **scale** across the cluster

Without a cluster manager, Spark would have to manage the cluster itself

## What?

A cluster manager:

- allocates computing resources across the cluster
- launches the processes required by Spark applications
- decides where executors run and how many resources they receive

## Examples

Spark can run on different cluster managers:

- **Spark Standalone** – built-in cluster manager
- **YARN** – commonly used in Hadoop ecosystems
- **Kubernetes** – container orchestration platform
- **Mesos** – general-purpose cluster manager

This flexibility allows Spark to run in **many different environments**

# Agenda

Distributed computing

Spark

Infrastructure

- Data

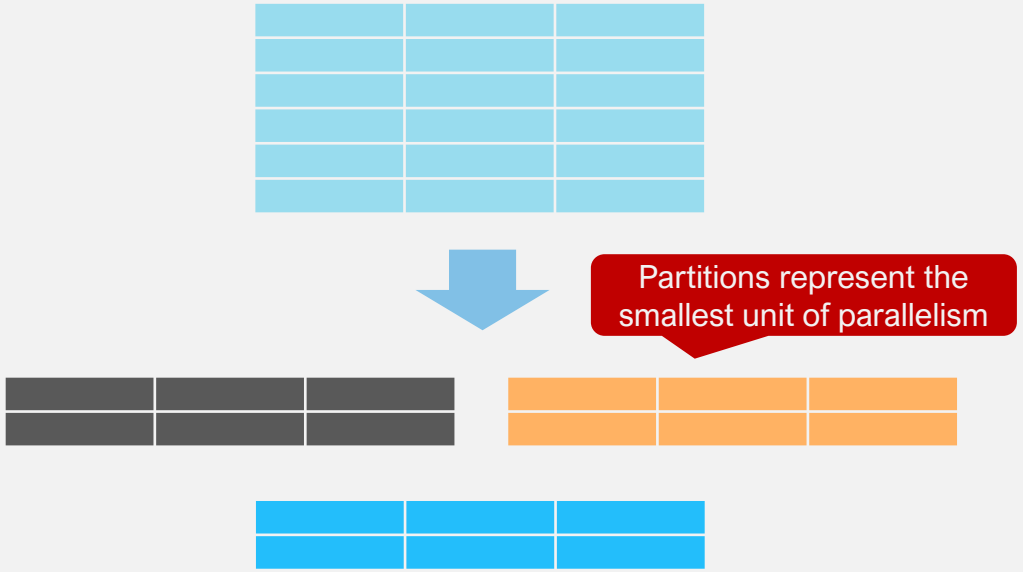
API

Runtime

Shuffle deep dive

# In such an architecture, data from the same dataset must be split among multiple machines. This is achieved by dividing the dataset in **partitions**

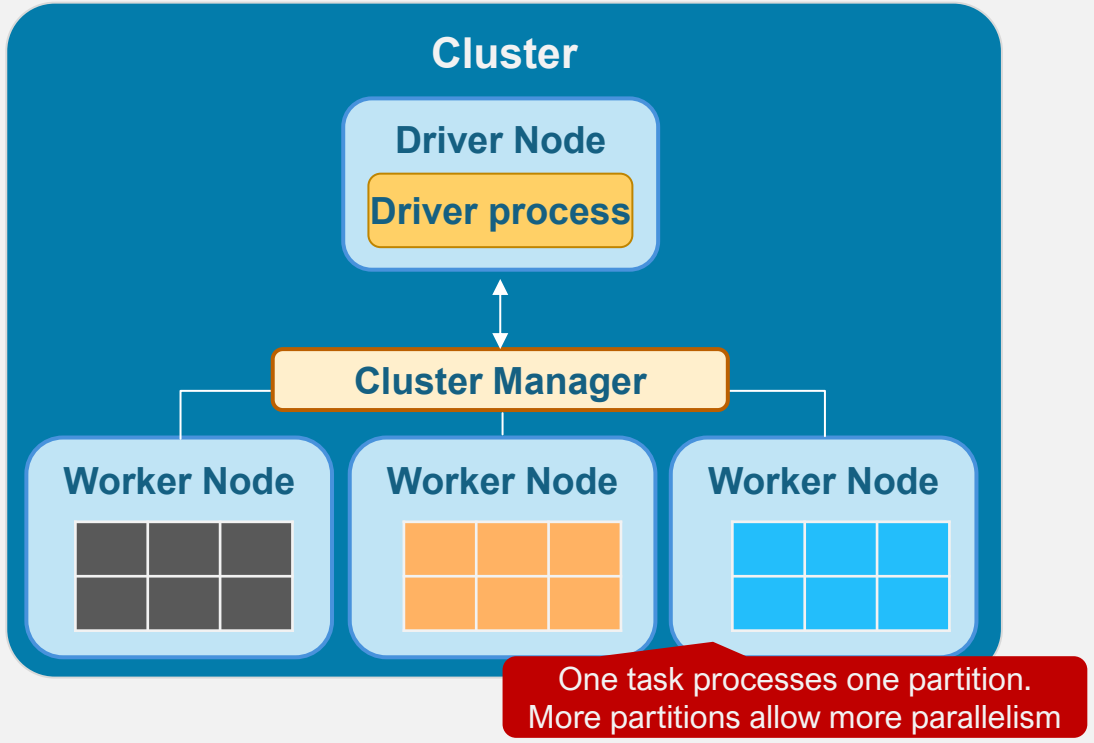
A **partition** is a logical chunk of a dataset that can be processed independently




Partitioning enables Spark to:

- process data **in parallel across machines**
- distribute computation across **executors**
- scale to **very large datasets**

Now Spark can assign each partition to an executor



# When data is spread across nodes, some records with the same key may end up on different machines

## Original dataset

| Key | Value |
|---|---|
| Pippo | 10 |
| Pluto | 12 |
| Mickey | 21 |
| Minnie | 8 |
| Pluto | 7 |
| Pippo | 16 |
| Pippo | 28 |
| Pluto | 21 |

## Partitioned dataset

**Worker Node**

| | |
|---|---|
| Pippo | 10 |
| Pluto | 12 |
| Mickey | 21 |

**Worker Node**

| | |
|---|---|
| Minnie | 8 |
| Pluto | 7 |

**Worker Node**

| | |
|---|---|
| Pippo | 16 |
| Pippo | 28 |
| Pluto | 21 |

**What if we want to compute, for example, average value by key? How can distributed data produce global results?**

# To compute global results, records must be **redistributed** across partitions so that related data is grouped together: meet the **shuffle**

## Starting partitions

**Worker Node**

| | |
|---|---|
| Pippo | 10 |
| Pluto | 12 |
| Mickey | 21 |

**Worker Node**

| | |
|---|---|
| Minnie | 8 |
| Pluto | 7 |

**Worker Node**

| | |
|---|---|
| Pippo | 16 |
| Pippo | 28 |
| Pluto | 21 |

## Redistributed partitions

**Worker Node**

| | |
|---|---|
| Pippo | 10 |
| Pippo | 16 |
| Pippo | 28 |

**Worker Node**

| | |
|---|---|
| Mickey | 21 |
| Minnie | 8 |

**Worker Node**

| | |
|---|---|
| Pluto | 21 |
| Pluto | 7 |
| Pluto | 12 |

Now we can compute average by key

**Shuffle: the redistribution of data across partitions**

# Agenda

Distributed computing

Spark

Infrastructure

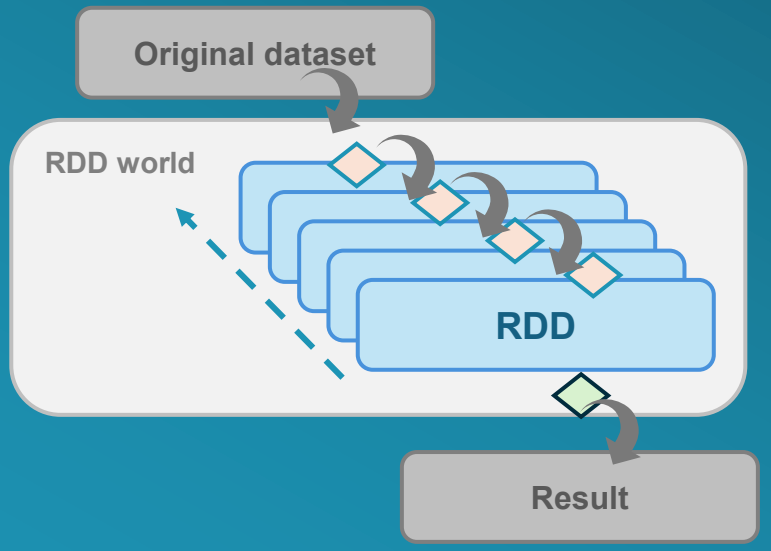
Data

- API

Runtime

Shuffle deep dive

# **4 key components** allow Spark to execute the distributed data workload




## RDD (Resilient Distributed Dataset)

RDDs are **collections of elements partitioned** across the nodes of the cluster that can be **operated** on in **parallel**

- They contain **immutable**, **lazily evaluated** plans that specify what operations to apply to data residing at a specific location to generate some output

## Transformations

Transformations are steps where the user describes (by coding) what operations to perform on data

- The **output** of a transformation is an **RDD**
- The execution of transformations will **not start until an action is triggered**

## Actions

Actions represent plans of how to manipulate rows and columns to compute the user's desired result

- Actions bring data **out of the RDD world** into some other storage system
- When we perform an action on an RDD, we instruct Spark to trigger the **evaluation of partitions** via the **DAG**

## Lineage (DAG)

The Lineage represents, by means of a directed acyclic graph (DAG), a sequence of transformations and actions created automatically by Spark, based on the dependencies between RDD transformations. When an action is run:

- The action triggers the **scheduler**, which **builds a DAG**
- Spark then evaluates the action by working backward to define the series of steps it must take to produce each object in the final RDD

Thanks to the DAG, Spark's **fault tolerance** is achieved, because each partition of the data contains the dependency information needed to recalculate the partition

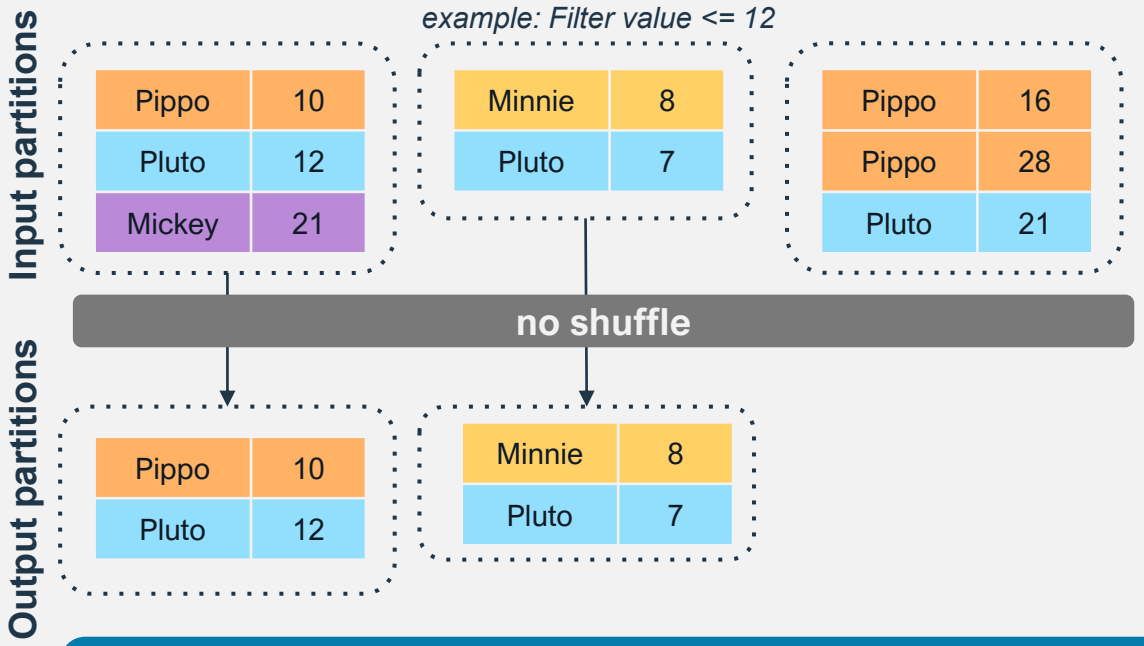
# Transformations are **narrow** or **wide** depending on whether each output partition depends on **one or multiple input partitions**

## **Narrow** transformations

`map`
`filter`

Each output partition depends on only **ONE** input partition.
So, each partition can be processed independently of the others.

- no data exchanged across partitions
- no shuffle

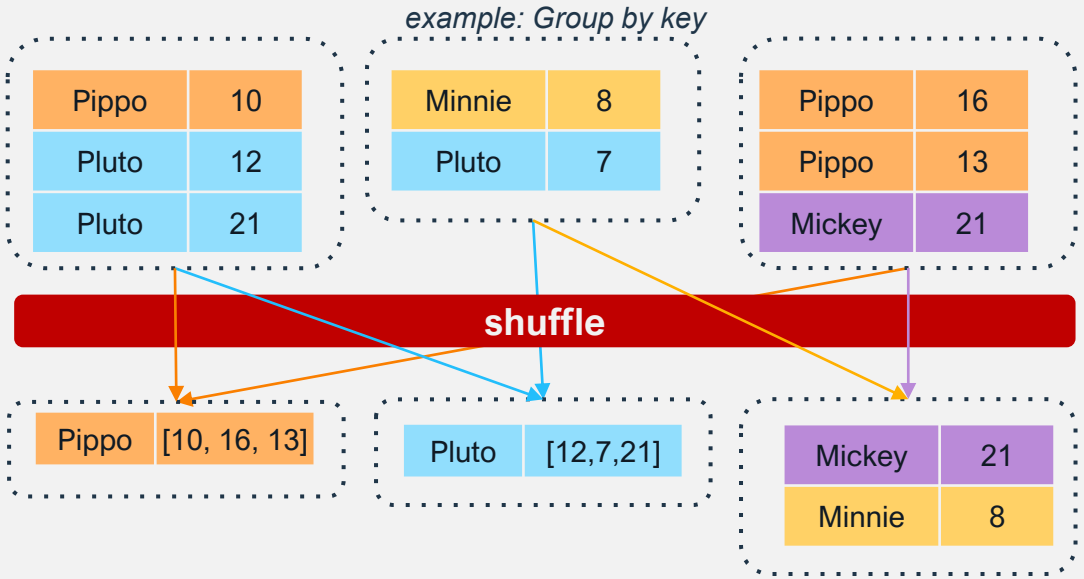



## **Wide** transformations

`groupByKey`
`join`

One output partition depends on **multiple** input partitions. So, data redistribution across partitions is required.

- data exchanged across partitions
- **shuffle required**




Narrow transformations preserve partitioning.
Wide transformations reshuffle the data.

# Agenda

Distributed computing

Spark

Infrastructure

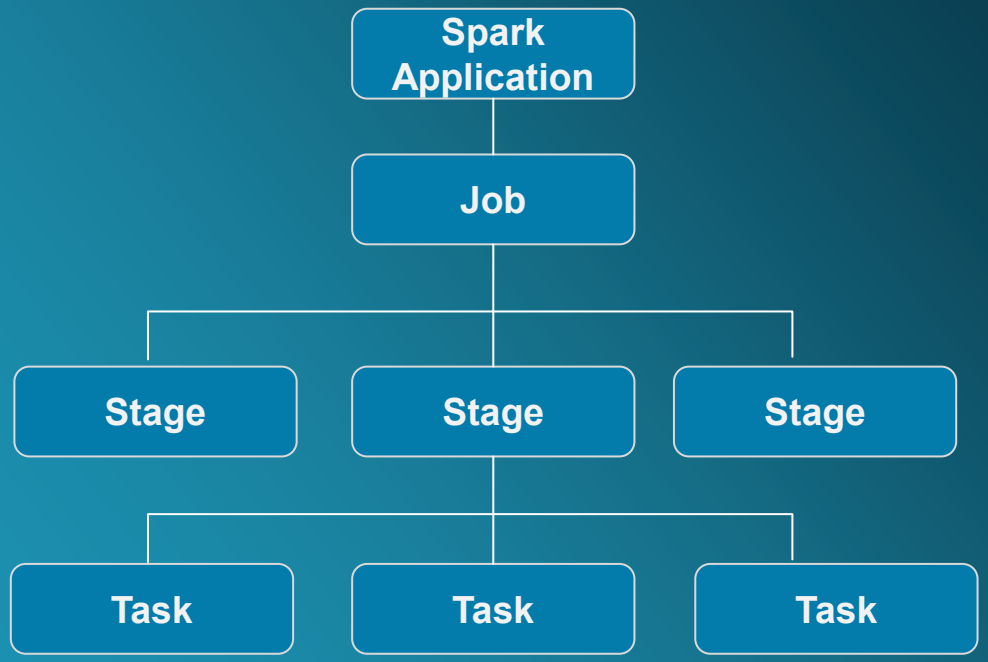
Data

API

- Runtime

Shuffle deep dive

# To break a computation into parallel work, Spark organizes execution into a hierarchy




**Task**

Smallest unit of work in Spark.
- Executed by an **executor**
- Processes **ONE data partition**

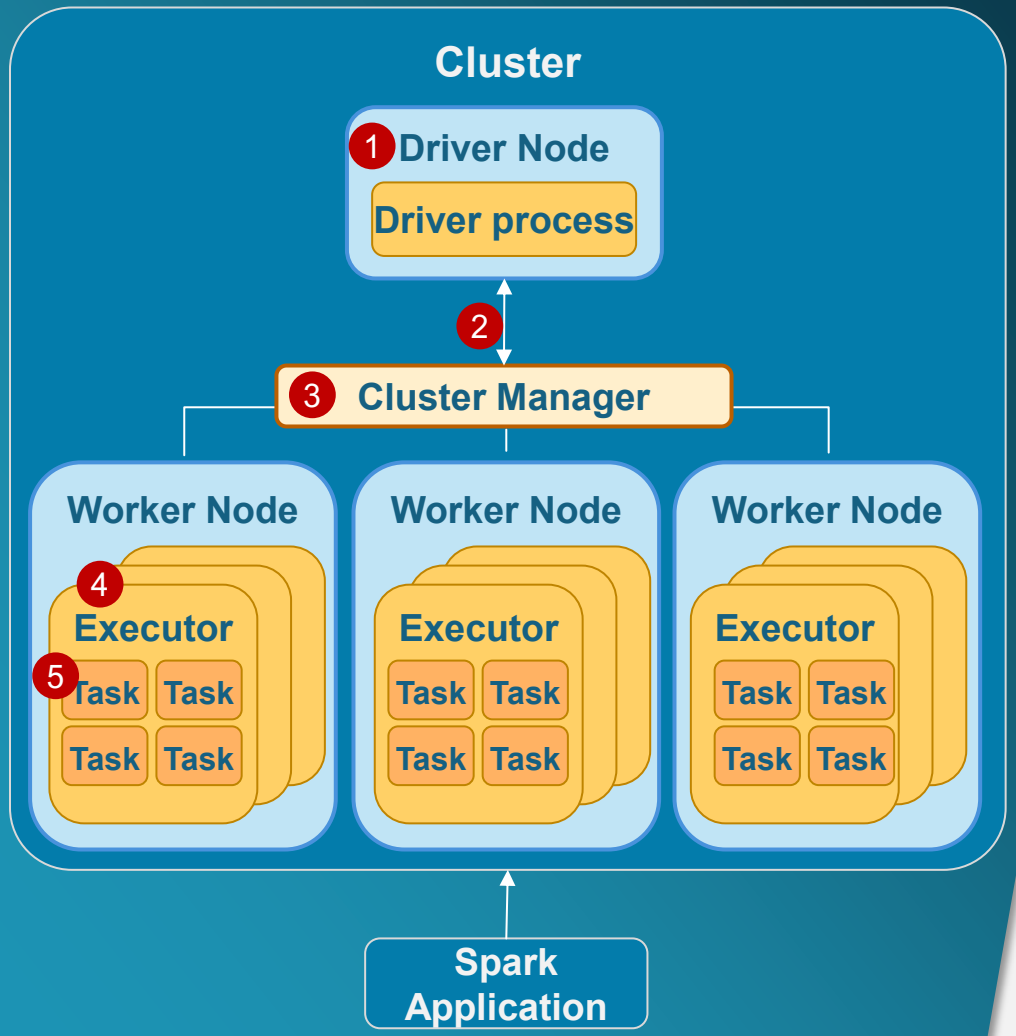
**Stage**

A group of **tasks** that can run in parallel.
- Created when Spark divides a job into execution steps
- Stages are separated by **shuffles** (wide transformations)

**Job**

A computation triggered by an **action**

# Let’s run a Spark application




1. User starts a Spark application through a Driver
2. The driver requests resources from the cluster manager
3. The cluster manager allocates resources and launches executors for the application
4. The driver distributes tasks to the executors
5. The executors execute the tasks by processing partitions and return the results to the driver

# Agenda

Distributed computing

Spark

Infrastructure

Data

API

Runtime

- Shuffle deep dive

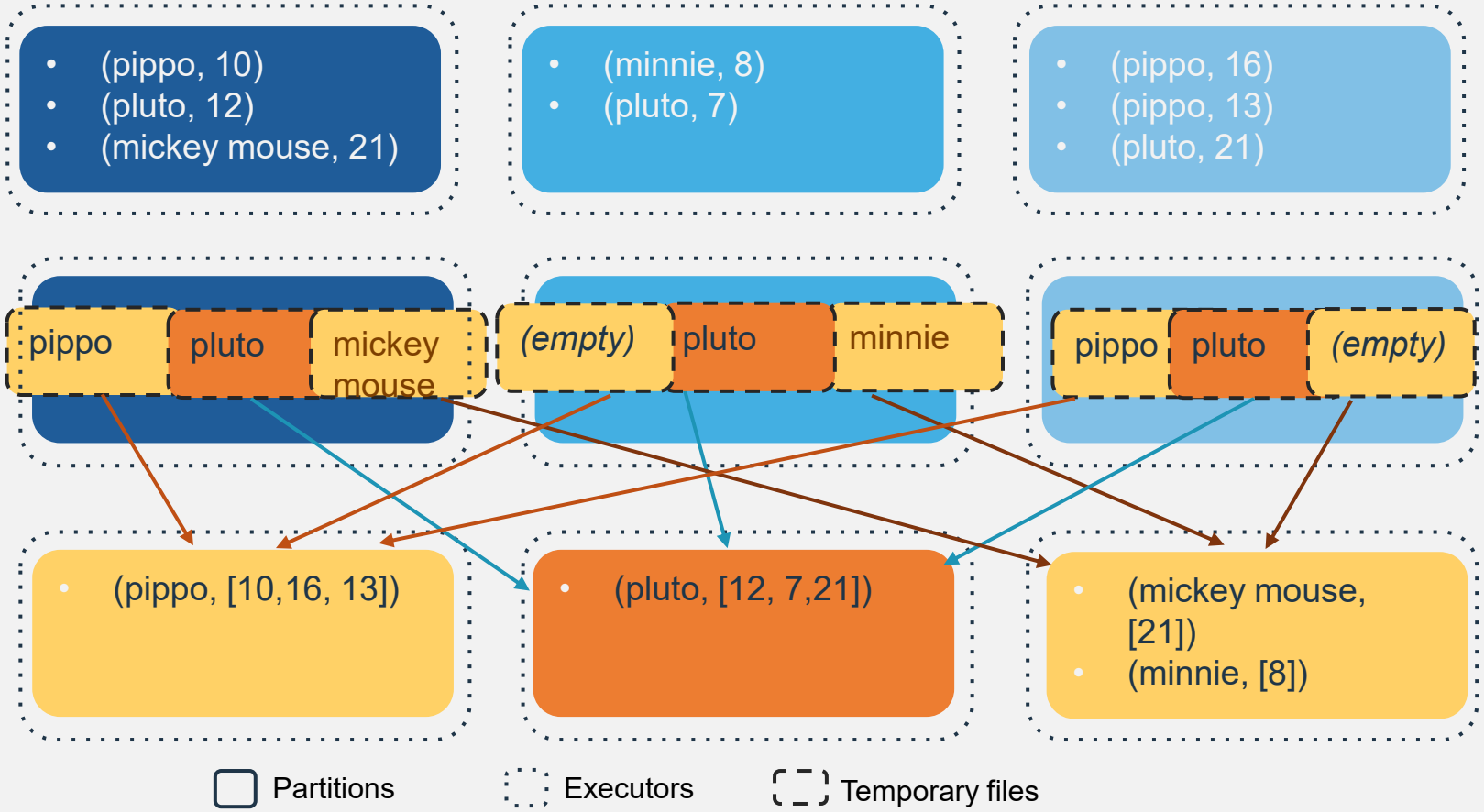
# There are 4 types of shuffle

# When a **transformation** requires data from **partitions other than itself** (e.g., sum all values in a column), **data** is **rearranged** between partitions through a **shuffle**

**Partitioning:** the original dataset is split into **partitions** to distribute work across the cluster and reduce memory requirements of each node

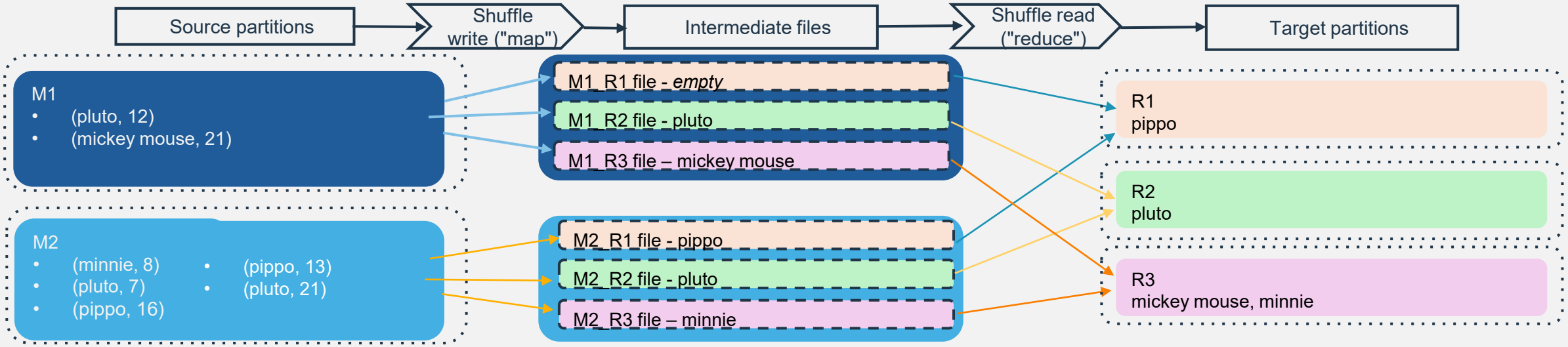
**Shuffle Write**: each exec takes its partition and **splits its data by key hash** into multiple blocks, **writing** them to **temporary intermediate blocks** (files on disk), one per dest. partition

**Shuffle Read**: each new partition reads data from all shuffle blocks, across all executors, via **disk I/O** (reading the temporary shuffle files), and **network I/O** (pulling them from all the machines), and assembles bew partitions




- **Shuffle is costly**: it involves copying data across executors and machines
- **Shuffle can be a bottleneck**: if key distribution is skewed, then more data will be placed in one partition than another, taking longer to be processed. As the next stage of processing cannot begin until all partitions are evaluated, overall results will be delayed

# Hash-based Shuffle




Each key is assigned to a specific reducer by means of a hash function and a *MOD # reducers* operation

| Mapper (source partition) | Key | **Assigned Reducer** = Hash(Key) MOD 3 |
|---|---|---|
| M1 | pluto | 1 |
| | mickey mouse | 2 |
| M2 | minnie | 2 |
| | pluto | 1 |
| | pippo | 0 |
| | pippo | 0 |
| | pluto | 1 |

**M=2** mappers

Each mapper creates a # of new files equal to the # of reducers (=1 file per reducer), e.g. *M1_R1, M1_R2,* etc.

Then, the mapper writes to the files according to the hash calculated in the previous step.

If in the initial partition there are no keys that hash to a specific number, the corresponding file will be empty, but will still be created. Why?

- Because reducers expect to receive one chunk of input from every mapper, even if it's empty
- This way, no reducer is left waiting and everything can be parallelized predictably

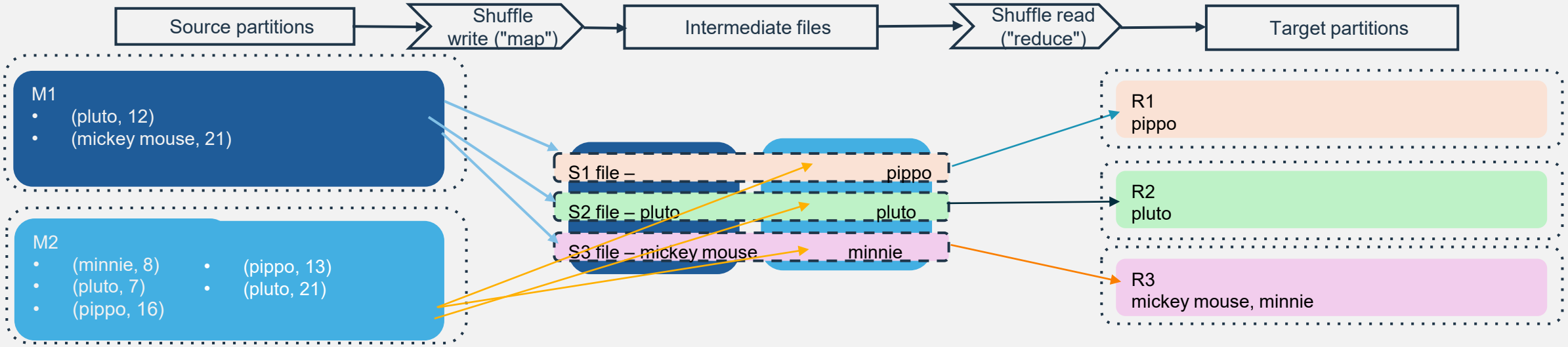
**MxR= 6** files written

Each reducer (i.e., target partition) reads data from exactly M files associated to its key

No need to have existence controls on files: each mapper will have emitted a file for each key, even if empty. This speeds up the shuffle read process

**R=3** reducers

# Hash-based Shuffle with Consolidation




Suppose to have the following shuffling problem:

- 46k partitions as source (46k "mappers"), to be reshuffled into 46k partitions as output (46k "reducers")

With a hash-based shuffle, we would need to write 2B files (46k x 46k). But the cluster can't write all these files in parallel! In fact, if my cluster is made of:

- 100 executors
- each with 10 cores

then we will have 1k cores of computing power.

Now, if 1 task (i.e., 1 "mapper") needs 1 core, then with 1k cores we can run max 1k "mappers" in parallel. And if each of these 1k "mappers" outputs a group of 46k files (the number of "reducers"), then, in a single run, we can write at most 46k * 1k = 46M fil

This means that to write all the 2B files, we will need to write 46 bunches of 46M each.
**Idea**: after having run the first job and having obtained the first 46M files, why don't we **reuse these files** instead of starting 46M new ones from scratch?

So, we continue sending the data into the same files, keeping the offset to identify which records come from which mapper

| Shared File | Keys from M1 | Keys from M2 |
|---|---|---|
| S1 | *none* | pippo |
| S2 | pluto | pluto |
| S3 | mickey m. | minnie |

With consolidation, Spark creates a pool of shared files, reused across waves of mappers, reducing the total file count to #cores × #reducers

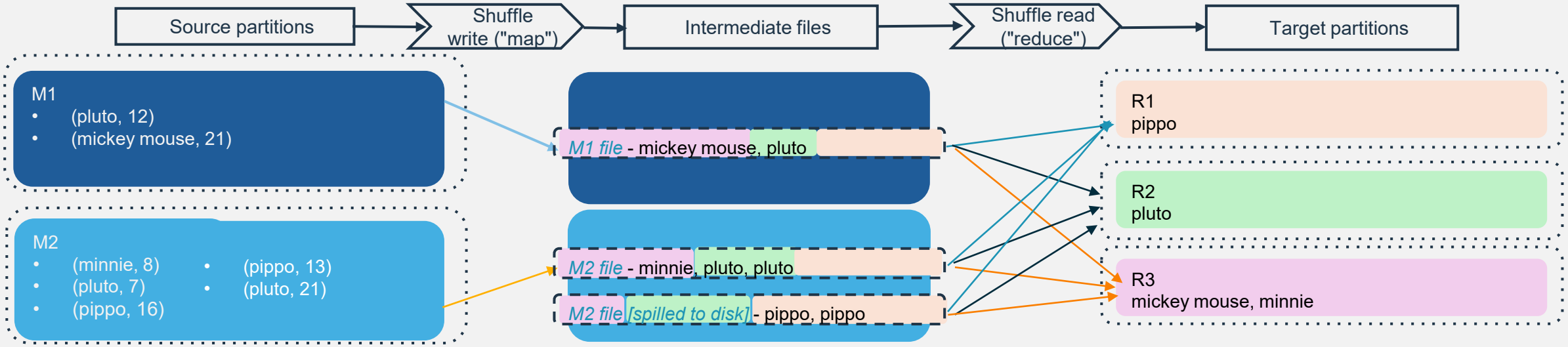
**M=2** mappers

**C=1** cores

**RxC= 3** files written

**R=3** reducers

# Sort-based Shuffle




Each mapper writes 1 file per mapper, **sorting the data** by **reducer ID**, and keeping an **offset index** of where each reducer's chunk is inside it.

As data gets sorted in the initial partitions, this kind of shuffle is useful for **map-side combine operations**

If data doesn't fit in memory, Spark starts spilling to disk. But before doing so, it sorts the spilled data.

Each mapper's spill is written to a separate file (e.g., M1 spill, M2 spill, etc.)

When the reducer comes in, Spark merges those spills in real-time using a priority queue (MinHeap-style)

When a reducer reads data from the output files, it goes through all files (including the spilled ones) and receives the **offset** of that file where needed data is stored

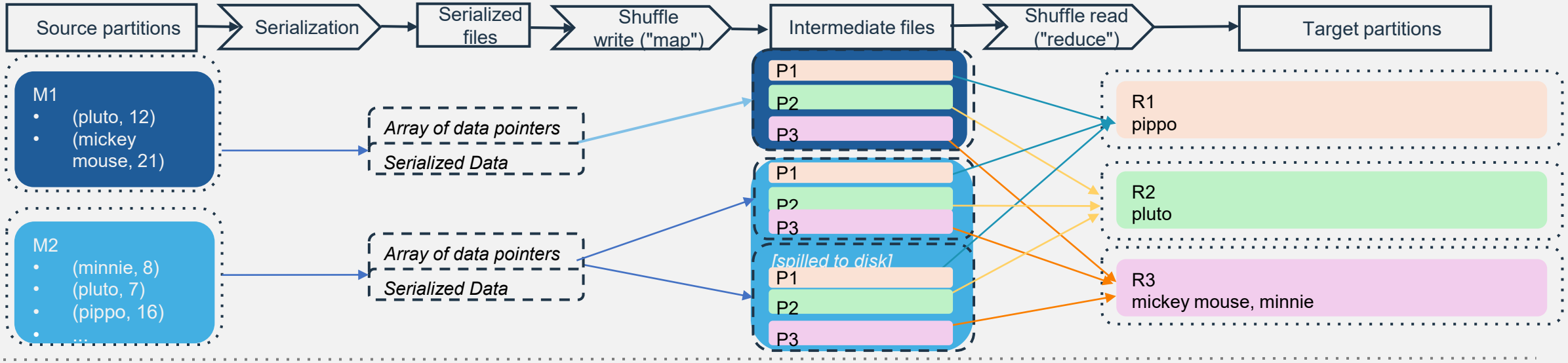
**M=2**
mappers

**M + spilled**
files written

**R=3**
reducers

Source:Spark Architecture: Shuffle | Distributed Systems Architecture

# Tungsten Sort or Unsafe Shuffle




Serializing means processing the records using their memory reference rather than the records itself, so without information about what the record contains (e.g., the key).

Spark usually works as follows:

- Deserialize data → process (e.g., shuffle) → reserialize → spill to disk

Tungsten shuffle is a technique designed to work with **serialized data** for the whole process:

- Keep the data serialized from start to finish
- Sort by just moving around light pointers, not the full records
- Use raw memory operations like pointer arithmetic
- Spill to disk without ever deserializing

After the serialization step, the shuffle follows the same logic of the sort-based shuffle. Note that this shuffle technique only works if:

- You **don't aggregate data** (no groupBy with sum/count/avg/etc.)
  - Why? There are two broad types of wide transformations:
    - Repartition-like: only care about which partition a record goes to (e.g. repartition, coalesce)
    - Aggregation-like: need to read/compare/merge data inside partitions (e.g. groupBy, reduceBy, aggregateByKey)

  In both cases Spark shuffles data — but the second case needs to know what's inside the boxes, so data must be unserialized
- Your serializer supports **byte relocation** (hello Kryo!)
- You have **<16 million partitions**
- You're **not stuffing 128MB+ per record**

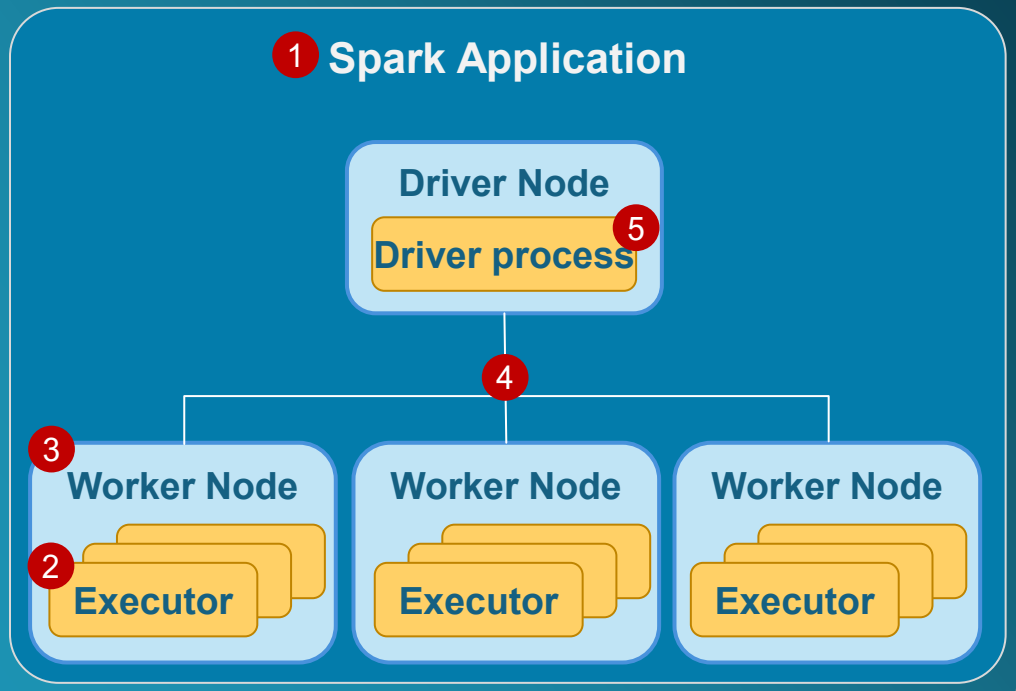
**M=2** mappers

**M + spilled** files written

**R=3** reducers

# Unused Slides

# Apache Spark is a distributed computing framework that works by coordinating computations across many machines in a cluster




Spark applications[1] run as independent sets of executor processes[2] on nodes[3] of a cluster[4], coordinated by a driver[5]

**1 Spark Application** — A program that runs computations on large datasets

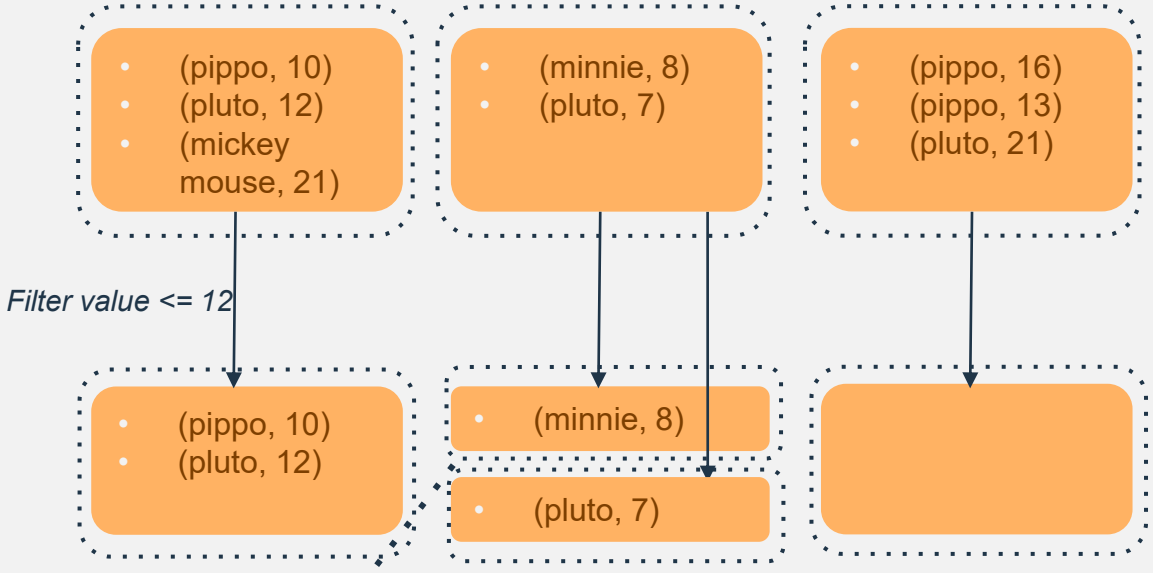
**4 Cluster** — A group of machines that work together to perform distributed computations

- **3 Node** — One individual server inside the cluster
  - **Worker Node** — A node capable of running executor processes
    - **2 Executor** — Independent process that runs computations on cluster nodes and stores data for your application
  - **Driver Node** — A node capable of running driver processes
    - **5 Driver** — Process that builds the execution plan for the applicartion

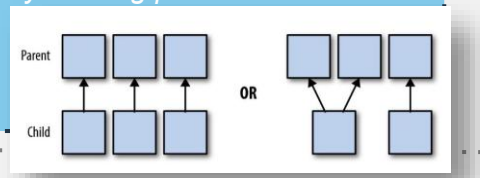
Source: https://www.friktoria.com/en/what-is-cluster-of-servers.html

# Transformations can be **narrow** and **wide,** based on whether the executors in different partitions need to communicate with each other

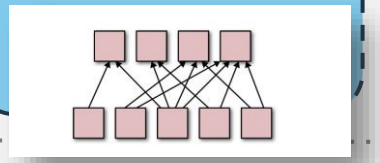
## **Narrow** transformations




*Input can also be split and sent to 2 different partitions, but can't be sent to an already existing partition*

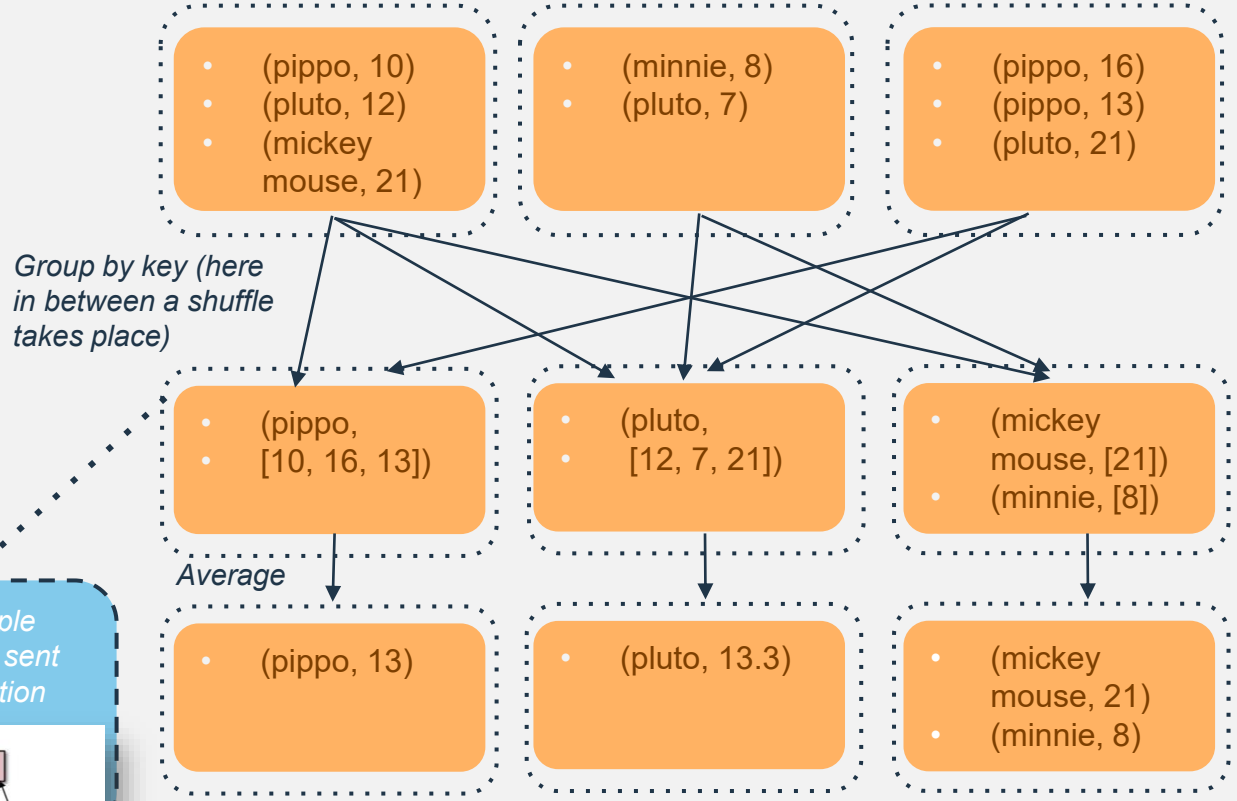



*Inputs from multiple partitions can be sent to the same partition*



- Executors **don't need to communicate** with each other
- Examples: map, filter

## **Wide** transformations




- Executors **need to communicate** with each other
- Examples: groupByKey, Join